

Security audit

Plume 1.1.0 — desktop application, Plume Vault API, and plume-md.com

8 October 2026 · static review and dynamic testing against production · CONFIDENTIAL

Scope	Method	Result
Desktop app (Electron, main + renderer)	Static review, dependency audit, live instrumentation	No issues of consequence
Plume Vault API (api.plume-md.com)	Static review, authenticated dynamic testing	No issues of consequence
plume-md.com (static site)	Static review, header and framing checks	1 medium — host headers
Third-party libraries that ship	npm audit, bundle inspection	1 medium — fixed in this pass

Four issues were found; two were fixed during the audit and are already in production. Two remain and both require access to the hosting dashboard, which the auditor does not hold. No issue allowed one account to reach another's data, and none permitted unauthenticated access.

Findings at a glance

#	Finding	Severity	Status
1	A KaTeX version with a published advisory was being shipped inside the application bundle	Medium	Fixed
2	The website serves no security headers; its sign-in page could be framed	Medium	Partly mitigated
3	Git identity values were passed to <code>git config</code> without a shape check	Low	Fixed
4	Build tooling carries eight advisories; none reaches a user	Informational	Accepted

Findings in detail

1 — A vulnerable KaTeX was shipping inside the bundle **MEDIUM FIXED**

What was wrong. `npm audit --omit=dev` reported no vulnerabilities, which is misleading for this project: every runtime library is declared as a *devDependency* because esbuild bundles it into the renderer. They all ship. Inspecting the built bundle's notices showed **two copies of KaTeX** going out — 0.19.0 required directly, and **0.16.47** pulled in by Mermaid. The second falls inside GHSA-238p-pmpm-9mq7, in which prototype pollution can bypass KaTeX's trust option.

Exploitability in context. Reduced by three existing controls: Mermaid runs at `securityLevel: 'strict'`, rendered SVG passes through a cleaner that strips interactive and top-layer elements, and the renderer's `script-src 'self'` policy blocks javascript: URLs. No working exploit was demonstrated. The finding is that a library with a known advisory was reaching users at all.

Fix applied. A single resolution is now pinned for every consumer:

```
"overrides": { "katex": "^0.19.0" }
```

The bundle now carries KaTeX 0.19.0 alone. Verified after the change by driving the real application: three KaTeX nodes and both Mermaid diagram types still render, with no console errors. No advisory now applies to any code that ships.

2 — The website serves no security headers **MEDIUM PARTLY MITIGATED**

What was wrong. Responses from `plume-md.com` carry no `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options` or `Referrer-Policy`. The practical consequence is that **/app.html — which carries the sign-in form — can be placed in a frame on any site**, which is the precondition for a UI-redress attack. The absence of HSTS also leaves a first visit open to downgrade.

The API responses are better: they set `no-store`, `nosniff` and `no-referrer`, and CORS is a real allowlist — a foreign origin receives no `Access-Control-Allow-Origin` at all.

What was done. The site is static and served by the host, not by the API, so headers cannot be set from the repository; `frame-ancestors` is explicitly ignored in a meta tag. Each page now refuses to be framed from its own head script, before paint. This is a mitigation and not a substitute for the header.

What remains for the owner. Set these at DigitalOcean App Platform (or in a Cloudflare transform rule) for the static component:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; frame-ancestors 'none'; object-src 'none';
base-uri 'self'
```

3 — Identity values reached `git config` unchecked **LOW FIXED**

What was wrong. When Git sync initialises a repository it may set a commit identity derived from the signed-in vault account. `git config <name> <value>` treats a value beginning with a hyphen as an option and offers no `--` terminator, so a hostile address shape would have been read as a flag. The value originates from the server, which validates addresses, so this was defence in depth rather than a live path.

Fix applied. The address is shape-checked before git is handed it. Confirmed by test: `--global`, `-x@y.com` and nonsense are refused; an ordinary address is accepted.

4 — Build-tooling advisories **INFORMATIONAL ACCEPTED**

Eight moderate advisories remain, all within electron-builder's dependency chain (@electron/get, app-builder-lib, global-agent, roarr, sprintf-js and siblings). None of this code is bundled into the application or executed on a user's machine; it runs only on the release builder. Recommended action is to track electron-builder updates rather than force a breaking downgrade.

Controls tested that held

The following were exercised directly, most of them against the production service with two purpose-made accounts that were deleted afterwards.

Control	How it was tested	Result
Cross-account document access (BOLA/IDOR)	Account A stored a document; account B requested it by the same path	Refused — 404
Unauthenticated vault access	Same request with no bearer token	Refused — 401
Path traversal	../../../../etc/passwd, encoded and doubled variants	Refused — 400/404
Credential stuffing	Twelve consecutive wrong passwords for one account	Limited after 9
Reset flooding	Eight consecutive reset requests	Limited after 4
Contact-form abuse	Twelve consecutive submissions	All limited
Account enumeration	Reset requested for a registered and an unregistered address	Identical answers
Secret storage in a Git remote	A remote containing ghp_... submitted to /git/status	Token stripped
CORS policy	Request bearing Origin: https://evil.example	No ACAO returned
Mail-header injection	Reply-To containing CRLF and a second recipient	Refused
Electron process isolation	Window preferences inspected in the running app	Sandboxed, isolated
Navigation and window escape	will-navigate, webview attachment, window open handler	All prevented
Secrets in version control	Pattern scan across tracked files in both repositories	None found

Observations, not findings

Mail credentials are held in the application database. They were placed there deliberately because the hosting environment was not reachable by the credential holder. This is defensible — anyone who can read that database already controls every account in it, and the Gmail grant is scoped to send only — but the environment remains the better home. Setting GMAIL_* on the application takes precedence automatically, after which the stored row can be removed with scripts/set-mail-credentials.js --clear.

Rate limiting is per-instance and held in memory. The code states this assumption plainly. It is correct for a single instance, and was observed to behave consistently under test. Should the API ever be scaled horizontally, the limits weaken proportionally and should move to shared storage.

The application is not code-signed. Windows SmartScreen and macOS Gatekeeper warn on first launch. Published checksums protect against a corrupted or substituted download but not against someone who controls the release itself. Signing remains the correct fix and is unchanged from previous releases.

Sign-up does not verify the address. The code-based flow exists server-side but no shipped client speaks it yet, so SIGNUP_REQUIRES_CODE is off and accounts are created unverified. Enabling it before those clients ship would stop account creation entirely; this has happened twice already.

Method and scope

Static. Manual review of the Electron main process, preload bridge and renderer; the vault API's routing, authentication, authorisation, storage and mail paths; and the website's scripts. Dependency analysis with npm audit on both projects, cross-checked against the built bundle's own notices, because the dependency manifest alone does not reveal what ships. Pattern scanning for credentials across all tracked files.

Dynamic. Authenticated and unauthenticated testing against the production API, including cross-account access attempts, traversal, rate-limit exhaustion and injection of a credential-bearing Git remote. Transport and header inspection of the website. Instrumentation of the running desktop application through the Chrome DevTools Protocol to confirm process isolation and renderer behaviour.

Out of scope. Infrastructure configuration at the hosting provider, the provider's own control plane, physical and social attack paths, and denial-of-service testing against production. No destructive testing was performed. Two accounts created for the authorisation tests were deleted; the five pre-existing accounts were untouched.

Prepared by CyberCrew. Findings 1 and 3 were remediated during the audit and the fixes are deployed. Finding 2 requires hosting-dashboard access to complete.